

Instagram AI Agent for [XOFlowers.md](https://www.xoflowers.md)

Built with ChromaDB + LLMs

Develop an **AI agent integrated with Instagram** that interacts with customers via **DMs**, understands their **intent**, and provides helpful responses such as **product recommendations, subscription flow, business info, and fictional payment handling**. The system will use **ChromaDB** for vector search and an **open-source LLM** for intent recognition.

Intent Definition:

The bot should handle user intentions:

1. **find_product**

The user wants to search for or get recommendations for a product, such as bouquets, gift boxes, or plants.

2. **ask_question**

The user asks a general question about the business, such as delivery hours, location, or return policy.

3. **subscribe**

The user wants to subscribe to a flower plan or receive updates and promotions from the business.

4. **pay_for_product**

The user expresses the intention to pay for a product. The bot should detect this and simulate a successful payment response.

The intentions above are only an example and they do not reflect the actual implementation. If the intent is not clearly recognized, the bot should use a **fallback prompt**.

Step 1: Collect Product Data from XOFlowers

- Website: <https://xoflowers.md/en/>
- Extract **all available products** manually or via scraping.
- Save the data in .json, .csv, or Python dict format:
- Fields: id, name, description, price, category

Step 2: Group Products into Categories

Example categories:

- **Bouquets, Boxes, Compositions, Plants, Gifts**

Each category should map to a separate ChromaDB collection.

Step 3: Store Data in ChromaDB

Set up **ChromaDB** locally.

Use sentence-transformers (e.g., all-MiniLM-L6-v2) to embed product descriptions.

Store embedded vectors in category-based collections.

Step 4: Intent Recognition with LLM

Use a local LLM (via **Ollama**) such as:

- Mistral 7B, LLaMA 3, TinyLlama

The LLM must:

- Detect intent from incoming messages
- Route messages accordingly (search, subscribe, ask, or simulate payment)
- Clarify ambiguous inputs when needed

Step 5: Instagram Integration via Graph API

- Create a **Facebook App** with Instagram Messaging access.
- Connect your **Instagram Business Account** to a Facebook Page.
- Use the **Graph API** to:
 - Listen for DMs
 - Send bot replies
 - Use Webhooks for real-time response

Docs:

- [Instagram Messaging on Graph API](#)

Step 6: Handle Each Intention

find_product

- Extract query
- Use ChromaDB vector search
- Respond with 3–5 product suggestions:

ask_question

- For questions like “What are your working hours?” or “Do you deliver in Chişinău?”
- Respond with pre-defined FAQ-style answers or infer with the LLM.

pay_for_product

- Simulate a payment confirmation:

Payment successful!

Your order has been confirmed.

We'll reach out shortly to finalize delivery

Step 7: Fallback Prompt

If no valid intent is detected, the bot should respond with a fallback message telling the customer it cannot help them and give some contact information.

Step 8: Censorship & Jailbreak Protection

Censorship: Block offensive terms using a basic keyword list or regex.

Jailbreak Detection:

- Filter messages like "ignore previous instructions"
- Respond:
 "I'm here to help with XOFlowers only . How can I assist?"

Step 9: Dummy Payment Flow

Goal:

Simulate a payment process when the user expresses intent to purchase a product.

Actions:

- Detect `pay_for_product` intent from user messages like "I want to buy this" or "How do I pay?"
- Ask the user which product they want to pay for if not already provided.
- Simulate a payment confirmation with a hardcoded success message.
- Log a fake order object in memory or store it in Redis/session for reference.
- Respond with a message like:

 Payment successful!

 Your order has been confirmed.

 We'll reach out shortly to finalize delivery.

Step 10: Cart Feature (Simulated)

Goal:

Allow users to add multiple products during a conversation before initiating the payment.

Actions:

- Create a temporary cart object per user (use Redis or in-memory dictionary).
- Support the following intents:
 - `add_to_cart` – e.g., “Add this bouquet to my cart”
 - `view_cart` – e.g., “What’s in my cart?”
 - `clear_cart` – e.g., “Empty my cart”
- Provide responses like:
 - "🌸 Spring Bloom added to your cart!"
 - "🛒 Your cart contains: 1x Spring Bloom, 2x Rose Box – Total: 1250 MDL"
- Allow user to say: “I’m ready to pay” → trigger Dummy Payment Flow using cart contents.

GOOD LUCK HAVE FUN!